

Introdução

O presente relatório analisa a implementação de um sistema básico de *matchmaking* desenvolvido em C++. O sistema gerencia uma fila de jogadores, permitindo operações essenciais como inserção, remoção, ordenação e agrupamento com base na similaridade *score* e *timestamp*.

Implementação dos Algoritmos de Ordenação

O sistema adota um critério de prioridade definido pelo método `comparePlayers`, que estabelece que jogadores com menor *score* têm prioridade e, em caso de empate, a precedência é dada ao jogador com o menor *timestamp*.

Insertion Sort

O algoritmo *Insertion Sort* foi implementado no método `sortByScoreInsertion`. A estratégia consiste em iterar pelo *array* a partir do segundo elemento, tratando a porção à esquerda do índice atual como um subvetor já ordenado. Para cada elemento, o algoritmo realiza comparações sucessivas com os elementos à sua esquerda, deslocando-os uma posição para a direita enquanto o critério de prioridade for satisfeito. Trata-se de uma ordenação *in-place*, não exigindo alocação de memória adicional durante sua execução.

Merge Sort

Para lidar de forma mais eficiente com conjuntos maiores de dados, o sistema implementa o *Merge Sort* através do método `sortByScoreMerge`. A recursão é o cerne desta abordagem, baseada no paradigma de divisão e conquista.

O método `mergeSortAux` divide recursivamente o *array* original em metades (esquerda e direita) até que cada subvetor contenha apenas um elemento (tamanho base, sendo um elemento único trivialmente ordenado). Após o alcance do caso base, a recursão retorna, e a função `merge` é chamada para combinar essas metades. A função `merge` utiliza um *array* dinâmico temporário para intercalar sequencialmente os elementos das metades esquerda e direita, respeitando a ordem ditada por `comparePlayers`, e por fim copia a porção ordenada de volta para o *array* original. O uso do *array* temporário pré-alocado (`temp`) no método principal evita alocações e desalocações repetitivas a cada chamada recursiva, otimizando drasticamente o processo.

Formação de Grupos

A formação de grupos (`formGroup`) aproveita diretamente o fato de a fila estar ordenada para utilizar uma abordagem semelhante a uma *janela deslizante*.

O algoritmo define dois ponteiros: `i` (início do possível grupo) e `j = i + K - 1` (fim do possível grupo). Ele itera linearmente pela fila, verificando apenas se a diferença de *score* entre o último e o primeiro elemento da janela (`players[j] - players[i]`) é menor ou igual à tolerância estipulada pelo `delta`. Se a condição for atendida, todos os jogadores intermediários necessariamente satisfarão o critério, e o grupo é formado com sucesso. Os jogadores selecionados são copiados para um novo *array* e removidos da fila original através de um deslocamento lógico dos elementos subsequentes.

Análise de Custo Computacional

- **Inserção (insert):** $O(1)$. A inserção ocorre sempre no final da porção preenchida do *array* dinâmico, acessando diretamente o índice livre através do contador `size` e incrementando-o.
- **Remoção (removePlayer):** $O(N)$. No pior cenário, o algoritmo realiza uma busca linear por toda a fila para encontrar o jogador alvo. Após a deleção local, precisa deslocar todos os elementos situados à direita uma posição para a esquerda, o que também tem custo associado a N .
- **Ordenação com Insertion Sort:** $O(N^2)$ no pior caso (array ordenado de forma inversamente proporcional à desejada) e no caso médio. No melhor caso (array já ordenado), sua complexidade cai para $O(N)$.
- **Ordenação com Merge Sort:** $O(N \log N)$ estrito garantido em todos os cenários (melhor, médio e pior), uma vez que o algoritmo sempre dividirá o vetor em metades de forma consistente. Requer custo $O(N)$ de complexidade espacial devido ao uso do *array* temporário no passo de junção.
- **Formação de Grupo (formGroup):** $O(N)$. O algoritmo percorre a fila linearmente e de maneira contígua para encontrar a janela com a tolerância desejada. O posterior deslocamento dos jogadores da fila principal também é feito em $O(N)$.
- **Recuperação de Jogadores (getWaitingPlayers):** $O(N)$. O sistema aloca um novo bloco de memória equivalente ao tamanho total atual e realiza a cópia elemento a elemento.

Comparação Experimental: Insertion Sort vs Merge Sort

Para medição de tempo, foi utilizada arquivo externo para geração de players aleatórios. A seguir os resultados.

A simulação consistiu na criação de *arrays* aleatórios ivididos em duas categorias populacionais: $N = 10.000$ e $N = 50.000$ elementos. A cronometragem limitou-se exclusivamente às rotinas de ordenação.

Algoritmo	Tempo Médio ($N = 10.000$)	Tempo Médio ($N = 50.000$)
Insertion Sort	~ 120 ms	~ 2.950 ms
Merge Sort	~ 3 ms	~ 16 ms

A tabela reflete diretamente a natureza das complexidades assintóticas dos métodos. O *Insertion Sort*, cujo custo no caso médio/pior é governado por $O(N^2)$, apresenta uma escalabilidade prejudicial. Ao multiplicarmos o tamanho da entrada por um fator de 5, o tempo de resposta sofreu uma elevação correspondente de aproximadamente 25 vezes.

Por outro lado, a eficiência da recursividade aliada à divisão e conquista torna o *Merge Sort* extremamente robusto. Exibindo um comportamento de ordem $O(N \log N)$, o custo computacional imposto pelo aumento populacional escala quase linearmente.